DAY 13 / CLASS REFRESHER

Recovery needs evidence

14 October 2026 · Harbour.Space Bangkok

Today combined an incident exercise with a workshop on product requirements and technical specifications. These notes recap confirmed class activity and explain the practices behind it. They add no new assignment.

What happened in the incident

The interface reported that spaces could not be loaded, and a request to the backend hung. Application logs said **“Could not connect to the database”**. Flurina acted as **incident commander (IC)**: the person coordinating the response. Pre from Purchase helped confirm the database problem.

15:15 - Spaces unavailable; the IC recorded P1 because all members were affected.
15:18 - Database reported back; recovery checking continued.
15:26 - Spacey and Payments answered and the spaces list loaded, but Access still returned an error. Flurina kept the phase at Monitoring and asked Access to investigate.
15:34 - All three answered normally; the IC recorded Resolved. [1]

These are the times written in the incident timeline. Its closing note reports a member-journey check, but does not give detailed booking, payment and unlock assertions. Treat this as the recorded classroom outcome, not a fresh independent test.

Three distinctions that matter

Symptom, evidence, explanation. The visible error describes the member’s problem. The connection error points towards a dependency. It does not, on its own, explain why that dependency became unavailable.

Dependency restored, service recovered. A running database did not establish that Access had recovered. In a later public clarification, Maksym reported manually restarting the Access container because it did not reconnect automatically. Restoring service and preventing recurrence are different pieces of work. [1]

Coordination, repair. The IC makes impact, current status and the next owner clear. The IC need not perform every repair. A useful update links an observation to a decision and the person taking the next action. [2]

Self-check

1. Why was “database back” insufficient evidence to close this incident?
2. What did the 15:26 observation justify doing next?
3. What would you record to make a member-journey recovery claim reproducible?

From customer problem to a checkable change

Confirmed workshop activity: Maksym presented prepared examples of a product requirements document (PRD) and a technical specification. Product Owners and Architects received the task of developing these documents for the earlier customer problem: **“My plans changed. I want to move my booking and understand what happens to my payment and access.”** Team documents are not reported as completed, approved or implemented.

Two documents, two questions

PRD: what should improve for the customer, and why? State the problem, who experiences it, the smallest useful outcome, scope and exclusions, and observable acceptance examples. Make unresolved product choices explicit. A list of screens or implementation tasks alone does not explain customer value. [3]

Technical specification: how will we deliver and check that outcome? Describe responsibilities, affected interfaces and data, failure behaviour, alternatives and trade-offs, and how the change will be tested and released. Connect each technical decision to a requirement. Keep detail proportional to the decision it helps someone make.

Product Owners and Architects work together: an engineering constraint can change the feasible scope, and a product choice can change the design. Record the agreement instead of allowing each team to assume a different answer.

Concrete PRD: move an existing booking

Customer outcome: Move an eligible booking to an available, differently priced slot in the same space, with the booking, agreed payment adjustment and access window describing the same result. Start with one agreed price direction; the teams must settle the policy before implementation.

User story: As a member whose plans changed, I want to move my booking so that I can use the space at the new time and understand the financial consequence.
Acceptance scenario to agree: Given an eligible booking and an available target slot, when I accept the quoted adjustment and the move completes, then the new slot is reserved, the old slot is released, one agreed adjustment is recorded and access matches the new window.

Product decisions still need evidence of agreement: eligibility, quote lifetime, unavailable slots, failed payment or access updates, retries, and what happens to the original booking while a move is pending. The linked technical specs explain how Frontend, Purchase, Payments and Access deliver and check those promises. This is the assigned feature scope, not evidence that it shipped.

Worked examples to reuse: Booking-label PRD and booking-label technical specification. These are fictional teaching examples: reuse their user story, numbered Given/When/Then scenarios, diagrams and links to checks; do not copy their feature policies into rescheduling.

Self-check

4. Which part of the example belongs in the PRD, and which in the technical spec?
5. What evidence would distinguish “document written”, “change deployed” and “customer problem improved”?

Answer cues: 1-2: Access still failed, so keep checking and give its investigation an owner. 3: Record the actions, expected and observed results, time and environment. 4: Agree the member outcome before selecting the mechanism. 5: A reviewed document, a verified running change and an observation of the customer outcome support different claims.

References and class record

[1] Spacey incident #277 - timeline, closure and later instructor clarification.
[2] Google SRE Workbook: Incident Response - roles, coordination and incident records.
[3] Atlassian: Product requirements documents - purpose, scope and shared understanding.
Class activity also reflects the instructor’s 14 October recap. References checked 14 October 2026.